

AI 使用说明情况（第三次挑战）

HW3-0102603133

1. 使用的 AI 模型与形态

WorkBuddy 内置 AI 编程智能体（agent 形态，具备本地文件读写、执行命令、以及通过 SSH/SFTP 操作远程 GPU 服务器的能力），底层为对话式大语言模型。全部交互在一个会话内完成。

2. 使用的提示词（原话摘录）

轮次	提示词 / 输入
1	上传《第一/二/三关挑战》三份 PDF，并给出服务器登录方式 <code>ssh -p 32566 root@connect.bjb2.seetacloud.com</code> （隐含指令：连接服务器，继续完成作业）
2	交互式选择：署名用学号 0102603133 ；target1 数据 在服务器上重跑一次

其余为智能体的自主执行，未再人工逐步下指令。

3. AI 如何帮助我学习与完成作业

- **环境与实验编排**：核查服务器状态（1×RTX 4090D、四个 SGLang 0.5.14 后端、conda 双环境），编写/修正 `run_round.sh`、`run_all_rounds.sh`，按课程要求顺序完成 A/B1/B2/C/D 五轮回放。
- **关键排障**：首轮 873/2048 条请求失败，AI 最初怀疑端口与容量问题，读取 `requests.csv` 原始错误后定位为 `ulimit -n=1024` 导致的 fd 耗尽（`OSError 24`），提升到 65535 后 B 组即全成功；又发现 A/C 组 队列排空时间超过压测端 300s 单请求预算，统一放宽为 900s 后整链重跑，五组均 2048/2048 成功。
- **target1 复现**：按挑战二规范实现 `measure_prefix_cache.py`（2048 共享前缀 + 64 独立后缀、32 条 × 并发 8、固定采样参数、组间 `flush_cache`），得到共享组命中率 97.06%、Prefill 仅 1,985 token 的对照组数据。
- **源码阅读定位**：在服务器安装的 SGLang v0.5.14 中逐一核对 `TokenizerManager.generate_request`、`Scheduler.event_loop_normal`、`RadixCache.match_prefix`、`cache_finished_req` 等关键函数所在文件，形成报告第 3 节的流程图与函数-文件表。
- **报告与数据一致性**：report.pdf 中所有数字由脚本直接从 `summary.json` / `requests.csv` 生成，避免转录错误。

4. 运用 AI 时是否被误导

有两处，且都靠回查原始数据纠正：

- **失败根因的初始误判**：AI 先把失败归因于“端口丢失/服务容量不足”，但 CSV 中的完整错误串显示 `port=8000` 且错误类型是 `Too many open files`，与端口无关。教训：必须让 AI 引用原始记录而不是凭摘要推断。
- **负载字段语义陷阱**：负载文件里的 `affinity_backend` 是课程方 按自己的哈希算法标注的“期望后端”，与 Ray Serve `ConsistentHashRouter` 的实际哈希环不一致，直接对比会得出“C 组只有 7.3% 请求落在亲和后端”的错误结论；改为使用命中率/后端分布等聚合指标后结论才成立。

除上述两处外，未发现 AI 编造数据或引用不存在的结论；报告中“与作业说明的两处 偏差”（单卡环境、fd/超时修复）均为 AI 主动如实标注。